



Nowrosjee Wadia College

Department of Computer Science

M.Sc. (Computer Science) - I

Subject : Principles of Programming Languages
Unit-1 Introduction

Prof. Sayli B. Patil

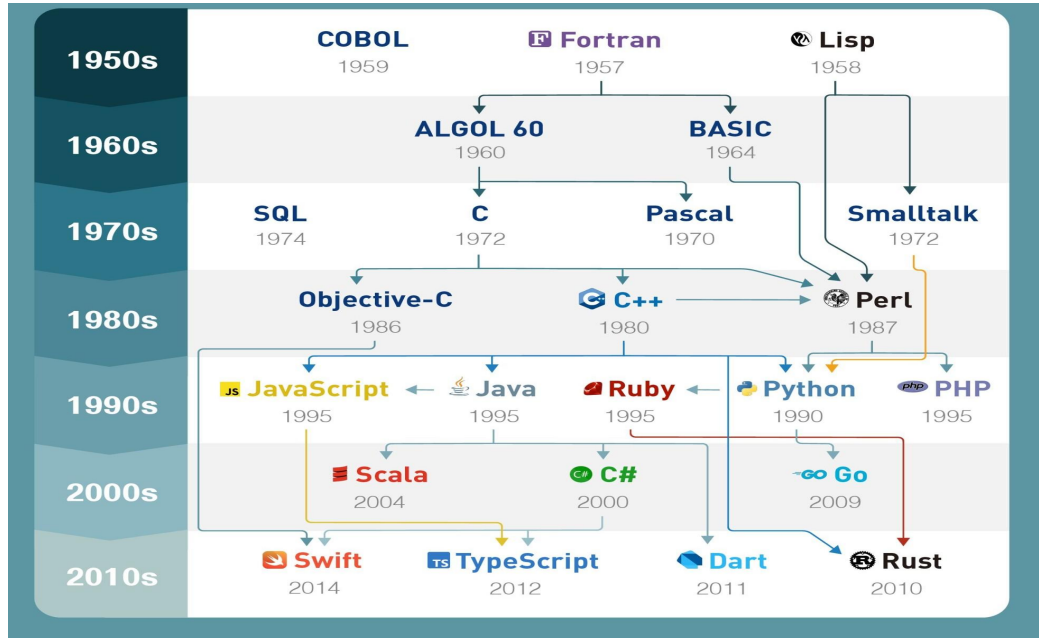
Topics to be covered

- ◆ **The Art of Language Design**
- ◆ **The Programming Language Spectrum**
- ◆ **Why Study Programming Languages?**
- ◆ **Compilation and Interpretation**
- ◆ **Programming Environments**

The Art of Language Design

✓ Why are there so many languages?

1. Evolution:
2. Special Purposes:
3. Personal Preference:
4. Expressive Power:
5. Ease of Use for the Novice:
6. Ease of Implementation:
7. Standardization:
8. Open Source:
9. Excellent Compilers:



The Art of Language Design



Why are there so many languages?

1. **Special Purposes** : Different applications has distinct requirements, and as a result, different programming languages might be more suitable for each task.
Ex. Web application, Scientific simulation, Embedded system, General Purpose.
2. **Personal preferences**: The multiplicity of languages ensures that developers can find a language that resonates with their individual approach to coding. Developers have the freedom to choose languages that align with their coding style and the problem they're trying to solve.

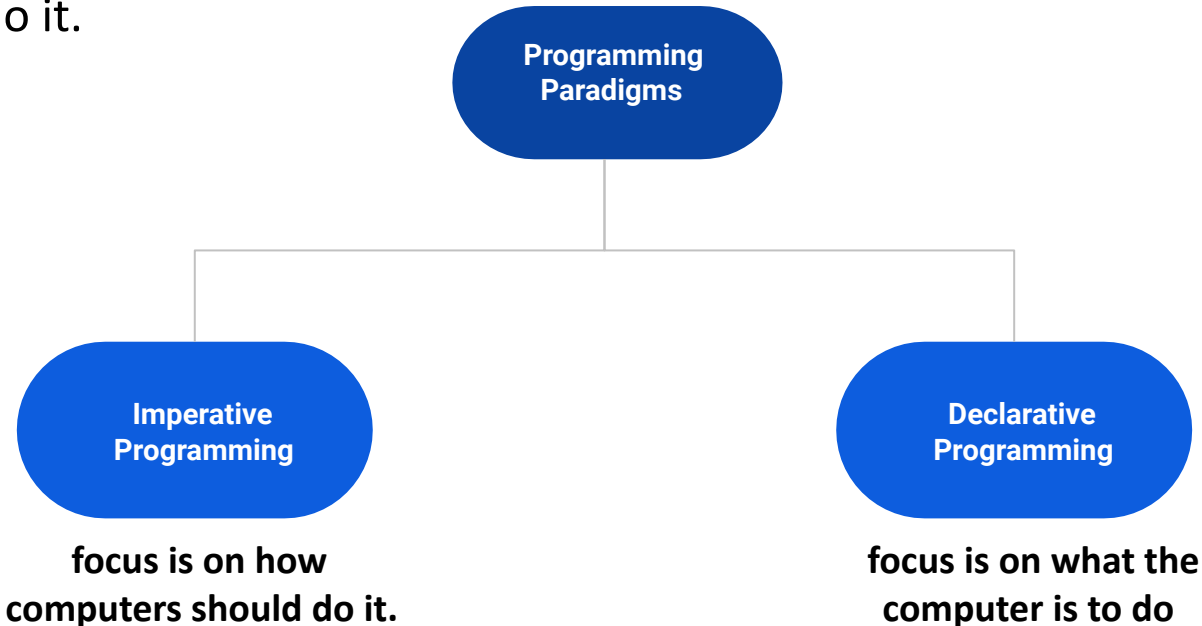
The Art of Language Design

Characteristics of good Programming language:

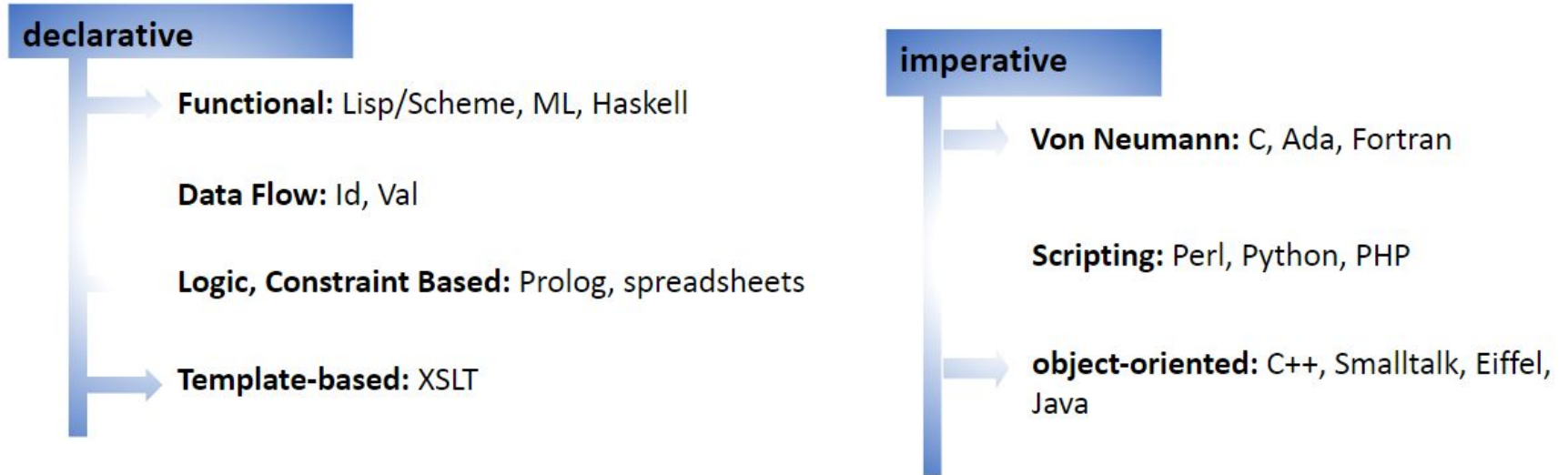
1. **Expressive power** : allows developers to create complex solutions efficiently
2. **Ease of use for the Novice**: encourages a broader range of people to learn and use the language
3. **Ease of implementation**: enables the language to be available on various platforms
4. **Standardization**: ensures consistency and compatibility
5. **Open source**: encourage community-driven development
6. **Excellent compilers**: excellent compilers and development tools enhance the overall development experiences.

The Programming Language Spectrum

- ✓ The many existing languages can be classified into families based on their model of computation, declarative languages, in which the focus is on what the computer is to do, and the imperative languages, in which the focus is on how the computer should do it.



The Programming Language Spectrum



Imperative Programming

- ✓ Imperative programming focuses on describing a sequence of steps that the computer must follow to achieve a desired outcome.
- ✓ It's like giving explicit commands to the computer on how to perform a task.
- ✓ This paradigm is closer to the machine's architecture and revolves around changing the program's state through assignments, loops, and conditional statements.
- ✓ Ex. Calculating the Sum of Even Numbers

```
total_sum = 0
for i in range(1, 11):
    if i % 2 == 0:
        total_sum += i
print(total_sum)
```

Declarative Programming

- ✓ Declarative programming, emphasizes describing what should be achieved rather than specifying the exact steps to achieve it.
- ✓ It focuses on expressing the logic and relationships between different parts of a program.
- ✓ It focuses on what the computer is to do. 'how' part is abstracted away
- ✓ Ex. Calculating the Sum of Even Numbers:

```
SELECT SUM(number)
FROM range(1, 10)
WHERE number % 2 = 0;
```

Why Study Programming Languages?

- ✓ Understand obscure features
- ✓ Choose among alternative ways to express things
- ✓ Make good use of debuggers, assemblers, linkers, and related tools.
- ✓ Simulate useful features in languages that lack them
- ✓ Make better use of language technology wherever it appears.

Why Study Programming Languages?

1. Understand obscure features (less commonly known features)
 - studying obscure features across multiple programming languages enables you to **identify common principles, appreciate differences, and gain a deeper understanding** of why these features exist and how they contribute to each language's uniqueness.
 - Ex. Python - Walrus Operator (`:=`)

```
while (line := input()) != "end":  
    print("You entered:", line)
```

Why Study Programming Languages?

2. Choose among alternative ways to express things
 - Knowing multiple programming languages provides you with the advantage of choosing alternative ways to express things based on the strengths and features of each language. It helps you in following ways:
 - Selecting the Right Paradigm
 - Leveraging Built-in Libraries
 - Performance Optimization
 - Conciseness and Clarity
 - Conforming to Conventions
 - Platform Compatibility

Why Study Programming Languages?

3. Make good use of debuggers, assemblers, linkers, and related tools.
 - These tools are crucial for software development, debugging, and optimizing code. By understanding how different languages interact with these tools, programmers can enhance their debugging and development processes. It can help in following ways:
 - Debugging Proficiency
 - Assembler Knowledge
 - Linking and Building
 - Interoperability
 - Optimization Insights

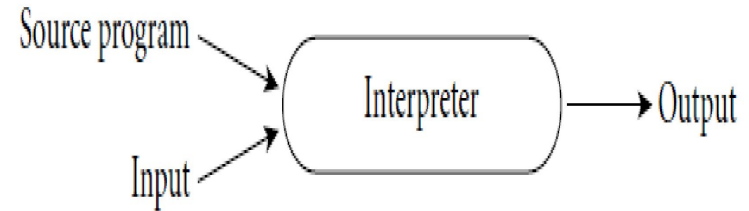
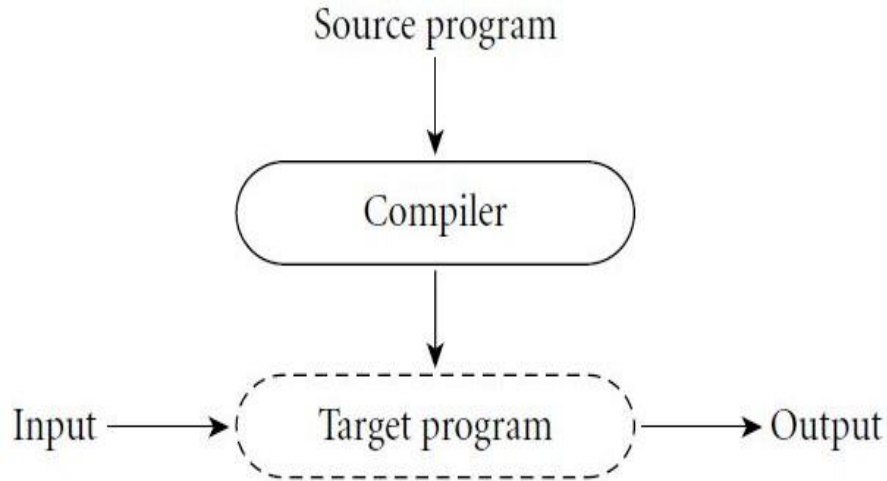
Why Study Programming Languages?

4. Simulate useful features in languages that lack them
 - Not all programming languages provide built-in support for every feature or concept, but by understanding different languages, programmers can employ creative workarounds to achieve similar functionality.
 - Cross-Language Compatibility
 - Expanding Language Capabilities
 - Handling Platform Differences
 - Optimization and Performance

Why Study Programming Languages?

5. Make better use of language technology wherever it appears.
 - Different languages often introduce innovative features and paradigms that advance the state of technology. Understanding these languages equips programmers to leverage these advancements effectively.
 - Keeping Up with Language Evolution
 - Leveraging Language-Specific Ecosystems
 - Innovating in Cross-Disciplinary Fields

Compilation and Interpretation



Compilation and Interpretation



Compiler :

- Translates high level source program into an equivalent target program.
- Compiler itself is a m/c lang. program.
- The compiler is in center of control during compilation and target code is in focus during execution.
- Supports early binding. Generates intermediate code i.e. .obj code (understandable to OS)
- Generates executable file. i.e. .exe code
- Fast, leads to better performance.
- Eg. C, C++, Pascal uses compiler.

Compilation and Interpretation



Compilation Process:

- During compilation, a high-level source program, written by a programmer, is transformed into machine language code by a compiler.
- This machine language code is often saved in a file format understood by the operating system, commonly referred to as object code.
- **Example:** Consider a C++ program called `my_program.cpp`. When compiled using a C++ compiler (e.g., `g++`), it generates an executable file, such as `my_program.exe` in Windows or `my_program` in Linux. This executable contains the machine code representation of the original C++ source code.

Compilation and Interpretation



Shift in Control:

- After compilation, the compiler's job is complete, and the control over the program shifts to the target program, the compiled machine code.
- This means the user can decide when to run the program, interact with it, and determine when it should terminate.
- **Example:** Once the `my_program.exe` file is created, the user can execute it at their convenience by running it from the command line or by double-clicking on it. At this point, the user decides when to start and stop the program's execution.

Compilation and Interpretation



Interpreter :

- Alternative to implement high level languages.
- Interpreter is responsible for program execution.
- Supports late binding (delaying decisions about program implementation until run time).
- More flexible and better diagnostics (error messages).
- No immediate code is generated.
- No executable code is generated. Therefore interpreter is required for every execution of program.
- Slow.

Compilation and Interpretation



Interpreter :

- The interpreter reads and executes statements one by one in the source code.
- Example: Consider Python, a widely used interpreted language. When you run a Python script (e.g., `my_script.py`), the Python interpreter reads and executes the code line by line. It's the interpreter that executes the program and responds to user input or runtime events.

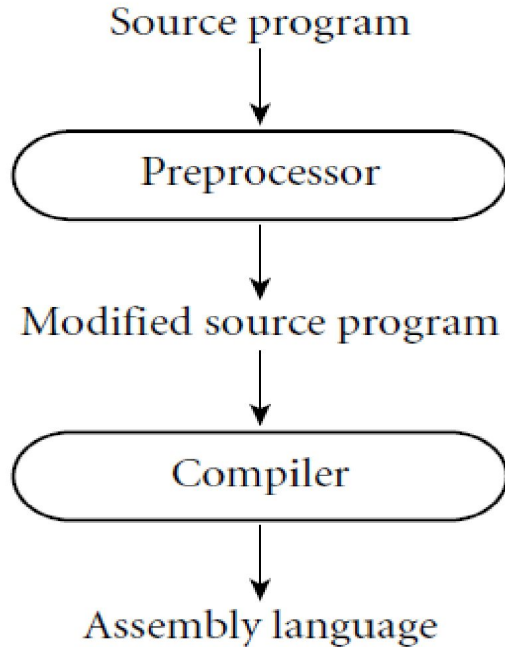
Compilation and Interpretation



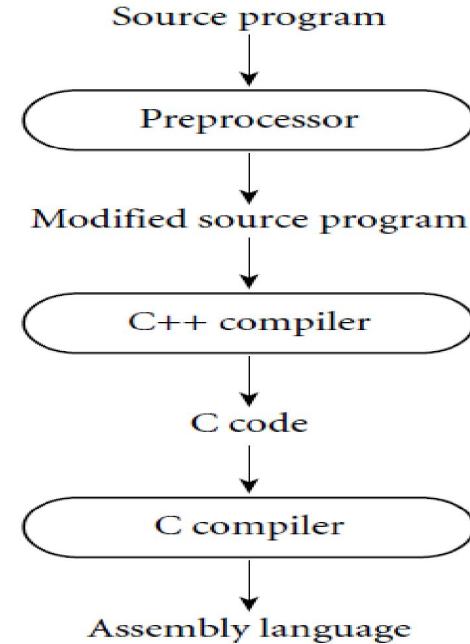
Preprocessor

- Most interpreted languages employ an initial translator (a preprocessor) that Preprocessing removes comments and white space, and groups characters together into tokens such as keywords, identifiers, numbers, and symbols.
- Compilers for C (and for many other languages running under Unix) begin with a preprocessor that removes comments and expands macros.
- The preprocessor can also be instructed to delete portions of the code itself, providing a conditional compilation facility that allows several versions of a program to be built from the same source.

Preprocessor



C Compiler with preprocessor



C++ Compiler

Preprocessor

- ✓ Some language implementation supports mixture of both compilation and interpretation.
 - Step 1 :source program is translated by the **translator** to intermediate program.
 - Step 2: Intermediate program and input is given to **virtual machine** which provides Output.
 - Eg. Java.
- ✓ Some Programming languages delay compilation until the last possible moment.
 - Eg. Prolog, Lisp invoke compiler **on the fly**, to translate newly created source code into m/c code or to optimize the code for a particular input set.
 - Java byte code.
 - First java implementations were based on byte-code interpreters.
 - Now more faster implementations using *just-in-time* compilers translates byte code into machine language immediately prior to execution.

Programming Environments

- ✓ The programming environment is a crucial factor in software development, encompassing everything from the tools and resources available to the development environment configuration.
- ✓ It plays a pivotal role in the entire software development lifecycle, from writing code to its execution.

Programming Environments

1. Writing Code:

- A well-configured programming environment provides a comfortable and efficient space for writing code.
- This includes integrated development environments (IDEs) or text editors with features like syntax highlighting, code completion, and version control integration.
- Example: Consider a software developer using an IDE like Visual Studio Code for writing Python code. The IDE offers real-time syntax checking, code formatting, and easy access to libraries and extensions, enhancing the developer's productivity and code quality.

Programming Environments

2. Debugging and Testing:

- A programming environment often includes debugging tools that help identify and fix issues in the code.
- These tools allow developers to set breakpoints, inspect variables, and step through code execution.
- Example: In a C++ development environment like Microsoft Visual Studio, developers can use the integrated debugger to identify and rectify bugs in their code by stepping through the execution, examining variable values, and pinpointing the source of errors.

Programming Environments

3. Libraries and Frameworks

- A programming environment typically provides access to libraries and frameworks that simplify common tasks and extend the functionality of programming languages.
- These libraries can save developers significant time and effort.
- Example: In web development, the JavaScript programming environment includes libraries like jQuery and frameworks like React and Angular. These tools facilitate front-end development by offering pre-built components and functionality.

Programming Environments

4. Compilation and Build Tools:

- A programming environment provides tools for compiling and building code into executable programs or libraries.
- These tools handle tasks like linking object files and resolving dependencies.
- Example: When working with Java, the Java Development Kit (JDK) includes tools like `javac` for compiling Java source code into bytecode and `jar` for packaging compiled code into JAR (Java Archive) files, simplifying the build process.

Questions

1. Why are there so many programming languages?
2. What makes a programming language successful?
3. Name three languages in each of the following categories: functional, object-oriented. Name two logic languages.
4. What distinguishes declarative languages from imperative languages?
5. Explain the distinction between interpretation and compilation. What are the comparative advantages and disadvantages of the two approaches?
6. Is Java compiled or interpreted (or both)? How do you know?
7. What is the difference between a compiler and a preprocessor?



Thank You...